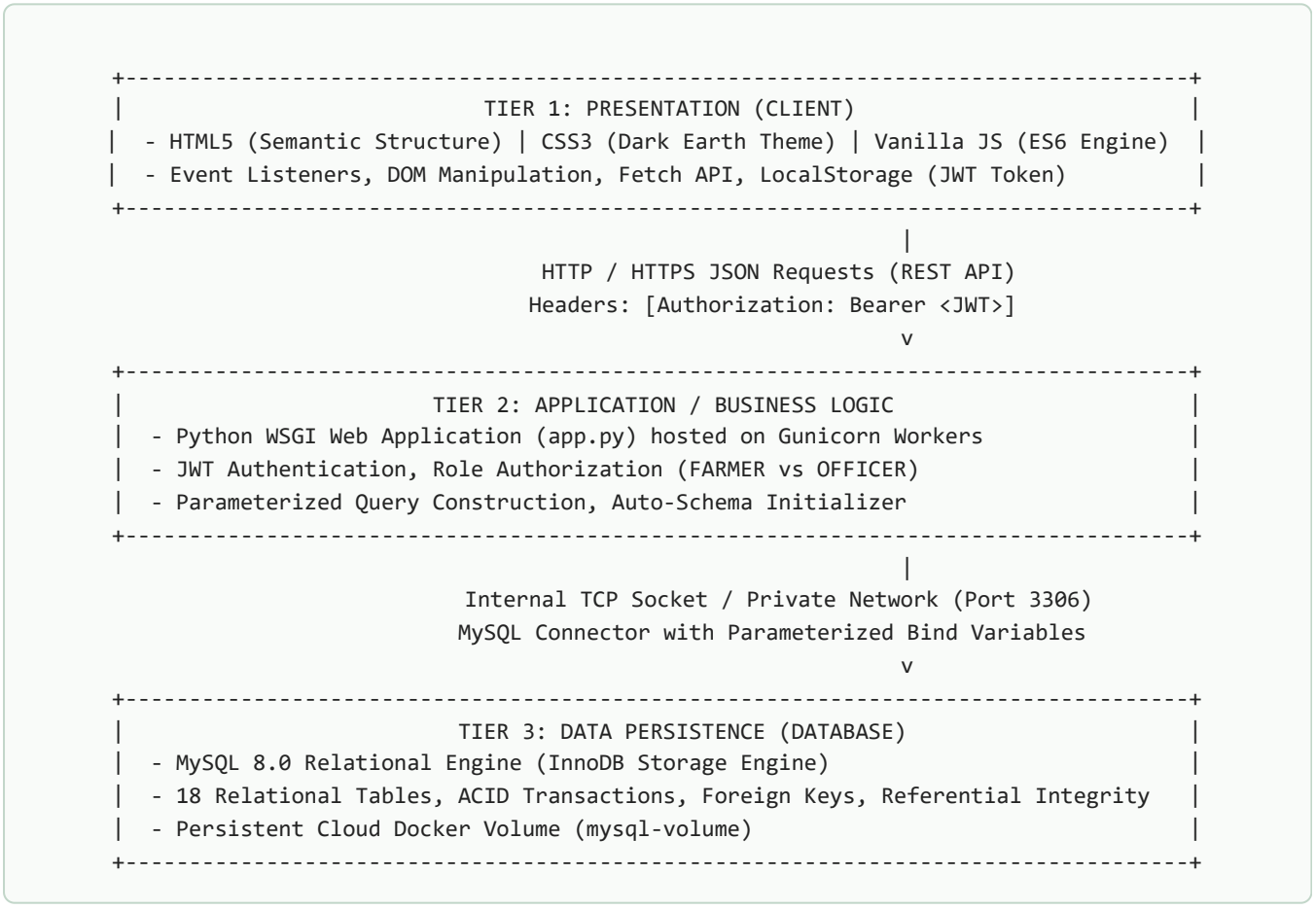
Agriculture Management System

Complete Technical Architecture, Data Lifecycle & Cloud Hosting Documentation

> **Document Purpose:** This document provides a complete, step-by-step breakdown of how the frontend interacts with the relational database (MySQL) through the application tier, followed by an in-depth walkthrough of the containerized cloud hosting architecture on Railway PaaS. Use this as your comprehensive reference for project defense, viva voce, and system presentation.

1. Executive Summary & High-Level Architecture

The **Agriculture Management System** is engineered as an enterprise-grade **3-Tier Web Application**. It strictly decouples user presentation from business rules and data persistence, ensuring security, scalability, and strict jurisdictional access control.



Layer	Technologies Used	Primary Responsibilities
Client Tier	HTML5, CSS3, Vanilla ES6 JavaScript	Renders UI, captures user input, client validation, sends asynchronous HTTP `fetch()` requests, updates DOM with responses.
Application Tier	Python 3.11, WSGI (PEP 3333), Gunicorn, MySQL Connector	API Routing, JWT verification, Role-based authorization, PBKDF2 password hashing, SQL statement generation, JSON serialization.
Database Tier	MySQL 8.0, InnoDB Engine	Guarantees ACID transactions, enforces primary/foreign key constraints, stores relations across users, officers, farms, crops, expenses, subsidies.
Cloud Hosting	Docker, Railway PaaS, Private Mesh Network	Container orchestration, automatic SSL termination, private internal DNS, persistent block volume storage.

2. The End-to-End Request & Data Lifecycle

To explain how the frontend connects to SQL, let us walk through a concrete user journey: **A Farmer logging in and registering a new Farm with land area and soil type**.

Step 1: User Action in Browser (Frontend DOM Event)

The user enters the farm name (`"Green Valley"`), area (`5.5` acres), and soil type (`"Alluvial"`) into the form on `farmer_dashboard.html` and clicks **"Save Farm"**.

- In `app.js` , an event listener intercepts the form submission:

```
// frontend/app.js
const farmForm = document.getElementById("farm-form");
farmForm.addEventListener("submit", async (event) => {
    event.preventDefault(); // Prevents default page reload

    const payload = {
        farm_name: document.getElementById("farm_name").value.trim(),
        area_acres: parseFloat(document.getElementById("area_acres").value),
        soil_type: document.getElementById("soil_type").value
    };

    await apiFetch("/api/farmer/farms", {
        method: "POST",
        body: JSON.stringify(payload)
    });
});
```

Step 2: HTTP Network Transmission (Fetch API & Security Header)

The browser's JavaScript runtime cannot and must never connect directly to MySQL (which would expose database passwords in the browser). Instead, it calls the helper function `apiFetch()` :

- It attaches the user's secret session token stored in `localStorage` :

```
// Headers sent over HTTPS:
POST /api/farmer/farms HTTP/1.1
Host: dbms-production-f338.up.railway.app
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjUsImV4cCI6MTc5MT...
Accept: application/json

{"farm_name": "Green Valley", "area_acres": 5.5, "soil_type": "Alluvial"}
```

Step 3: Web Server & WSGI Gateway Routing

1. **Railway Edge Router:** Accepts the HTTPS request, handles SSL/TLS termination, and forwards plain HTTP internally to port `8000/8080` where the container listens.
2. **Gunicorn WSGI Master & Worker:** Gunicorn manages a pool of pre-forked worker processes. A worker picks up the connection and hands the standard WSGI environment dictionary `environ` to

Python's `application(environ, start_response)` function.

3. **Endpoint Dispatching:** In `backend/app.py`:

```
# backend/app.py
path = urlsplit(environ.get("PATH_INFO", "/")).path
method = environ.get("REQUEST_METHOD", "GET").upper()

if path == "/api/farmer/farms" and method == "POST":
    user = authenticated_user(environ, "FARMER") # Verifies JWT & Role
    payload = parse_json_body(environ)
    result = handle_create_farm(payload, user)
    return json_response(start_response, 201, result)
```

Step 4: Authentication & Role-Based Authorization

Before any SQL query touches the database, the server runs `authenticated_user()`:

- Extracts the token from `HTTP_AUTHORIZATION` header.
- Verifies HMAC-SHA256 signature against `APP_SECRET_KEY` to confirm it was not tampered with.
- Checks token expiration (`exp > time.time()`).
- Verifies that the user's role in the database is strictly `FARMER`. If an officer or unauthenticated visitor attempts this endpoint, execution immediately aborts with `HTTP 403 Forbidden`.

Step 5: Database Connection & Parameterized SQL Execution

Once authorized, the server calls the business logic function:

```
# backend/app.py
def handle_create_farm(payload: dict, user: dict) -> dict:
    # 1. Fetch farmer_id linked to user_id
    farmer_id = get_farmer_id(user["user_id"])

    # 2. Acquire safe database connection
    with database_connection() as connection:
        cursor = connection.cursor()

        # 3. Parameterized INSERT Query (prevents SQL injection)
        sql = """
            INSERT INTO Farms (farmer_id, farm_name, location, area_acres, soil_type, irrigation_typ
            VALUES (%s, %s, %s, %s, %s, %s)
        """
        params = (
            farmer_id,
            payload["farm_name"],
            payload["location"],
            payload["area_acres"],
            payload.get("soil_type"),
            payload.get("irrigation_type")
        )
        cursor.execute(sql, params)
        connection.commit() # Enforces ACID Durability

        new_farm_id = cursor.lastrowid
        return {"message": "Farm registered successfully", "farm_id": new_farm_id}
```

> **Vital Viva Point — Why Parameterized Queries?** Notice the use of `%s` placeholders instead of Python string formatting (`f"INSERT ... {payload['farm_name']}"`). With parameterized queries, MySQL treats parameters strictly as **literal data**, never as executable SQL commands. This completely prevents **SQL Injection attacks** (such as `"'; DROP TABLE Farms; --"`).

Step 6: MySQL Transaction & Constraint Verification

Inside MySQL's InnoDB engine:

1. **Foreign Key Check:** MySQL validates that `farmer_id` exists in the `Farmers` table. If the ID does not exist, MySQL rejects the transaction with a foreign key violation.
2. **CHECK Constraint:** The table defines `CHECK (area_acres > 0)`. If a user entered `-5`, MySQL rejects it immediately.
3. **Transaction Commit:** The row is written to the redo log and committed with ACID compliance.

Step 7: Response Serialization & DOM Update

1. The Python server packages the result dictionary into JSON, encodes it to UTF-8 bytes, and sends response headers (`HTTP/1.1 201 Created`, `Content-Type: application/json`).
2. The browser's `await fetch()` resolves. The JavaScript engine parses the JSON:

```
// frontend/app.js
const data = await response.json();
showToast("Farm added successfully!", "success");
// Immediately reload farms table dynamically without full page refresh
loadFarmerFarms();
```

3. Database Schema & Relational Architecture

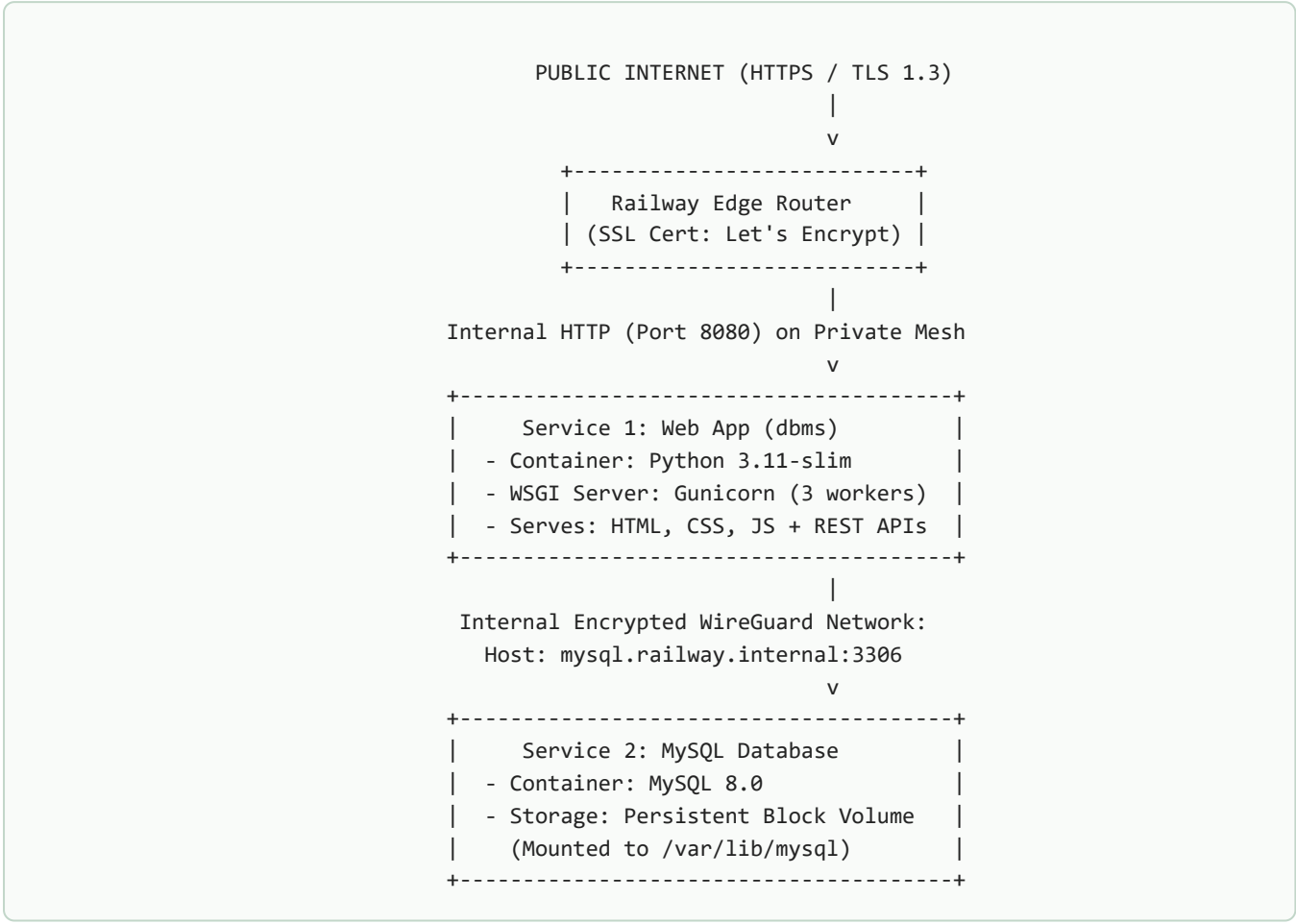
The system consists of **18 interrelated base tables** and **1 directory view**, maintaining 3NF (Third Normal Form) normalization:

Table Name	Primary Key	Foreign Keys & Dependencies	Core Role
`Users`	`user_id`	None (Root table)	Stores authentication credentials (email, hashed password, role).
`Agriculture_Offices`	`office_id`	None	Krishi Bhavans / Regional agricultural administrative offices.
`Panchayaths`	`panchayath_id`	`office_id` → `Agriculture_Offices`	Gram Panchayaths falling under specific Krishi Bhavan jurisdictions.
`Officers`	`officer_id`	`user_id` → `Users`, `office_id` → `Offices`	Agricultural officers assigned to specific Krishi Bhavan offices.
`Farmers`	`farmer_id`	`user_id` → `Users`, `panchayath_id` → `Panchayaths`	Farmer profiles bound to their local Gram Panchayath.
`Farms`	`farm_id`	`farmer_id` → `Farmers`	Individual agricultural land parcels owned by farmers.
`Crops`	`crop_id`	None (Master Catalog)	Approved crop catalog (strictly configured for Rice and Wheat).
`Farm_Crops`	`farm_crop_id`	`farm_id`, `crop_id`	Cultivation records: Season, planting date, harvest date, area, status.
`Expenses`	`expense_id`	`farm_id`, `farm_crop_id`, `rental_id`, `energy_id`	Farm expenditures (Seeds, equipment rental, energy, irrigation).
`Income`	`income_id`	`farm_crop_id`	Crop sales with automatically computed virtual column `total_income`.
`Subsidies`	`subsidy_id`	None (Master Catalog)	Government schemes (Subhiksha Keralam, Organic Farming, Machinery).
`Farmer_Subsidies`	`farmer_subsidy_id`	`farmer_id`, `subsidy_id`, `officer_id`	Application lifecycle: PENDING → APPROVED → REJECTED with strict officer jurisdiction checks.

> **Jurisdictional Approval Enforcement:** When an officer attempts to approve a subsidy (`POST /api/officer/subsidies/review`), the backend executes an inner join verifying that the applicant farmer's Gram Panchayath belongs to the officer's designated Krishi Bhavan: `p.office_id = o.office_id` . If an officer from Thrissur attempts to approve a Palakkad farm application, the system blocks the transaction with `HTTP 403 Jurisdictional Violation` .

4. Cloud Hosting Architecture (Railway PaaS & Docker)

The system is hosted in the cloud on **Railway** using modern containerization and micro-networking.



1. Docker Containerization (`Dockerfile`)

Instead of relying on server-specific configurations, the web application is packaged inside an isolated Linux container:

```
FROM python:3.11-slim
WORKDIR /app
COPY backend/requirements.txt ./backend/requirements.txt
RUN pip install --no-cache-dir -r backend/requirements.txt gunicorn
COPY . .
EXPOSE 8000
CMD ["sh", "-c", "gunicorn --bind 0.0.0.0:${PORT:-8000} --workers 3 backend.app:application"]
```

- **Port Binding:** Uses `${PORT:-8000}` because Railway dynamically assigns an internal port (such as `8080`) at boot time.
- **Workers:** `--workers 3` runs three independent OS worker processes in parallel to handle concurrent user requests without blocking.

2. Private Mesh Networking (Security by Isolation)

A critical architectural decision in this project: **The MySQL database is NOT exposed to the public internet**.

- Railway creates a private internal DNS: `mysql.railway.internal`.
- Only containers within the same Railway project canvas can communicate over port `3306`.
- Public attackers on the internet cannot port-scan or brute-force the MySQL database because no public IP or public port exists for it.

3. Persistent Volume Storage (`mysql-volume`)

By default, Docker containers are ephemeral (their files disappear when restarted). To guarantee that user records, registered farms, and applications are never lost:

- Railway mounts a persistent cloud block storage volume to `/var/lib/mysql`.
- Even during server reboots, git updates, or redeployments, MySQL data files remain safe and intact on persistent disk.

4. Self-Initializing Database Engine (`init_database_if_needed()`)

To solve the challenge of provisioning a clean database in the cloud without manual SQL imports, our backend incorporates an intelligent auto-migrator:

```
def init_database_if_needed() -> None:
    config = get_database_config()
    if not config.get("user"):
        return
    connection = mysql.connector.connect(**config)
    cursor = connection.cursor()
    cursor.execute("SHOW TABLES LIKE 'Users'")
    if not cursor.fetchone():
        # Executes backend/schema.sql (18 Tables + Views)
        # Executes backend/seed_catalog.sql (Krishi Bhavans, Panchayaths, Crops, Subsidies)
        connection.commit()
```

On its very first boot or on the initial health check, the application detects that the database is empty, runs the entire schema, populates reference catalogs, and transitions itself to fully ready.

5. Viva Voce & Presentation Defense Guide

Use these standard questions and exact technical answers to confidently present this architecture to your examiners or evaluators.

Q1: Can frontend JavaScript connect directly to MySQL? Why or why not?

Answer: No. In web browsers, JavaScript runs in a client-side sandbox and does not have raw TCP socket access to talk directly to MySQL protocol on port 3306. More importantly, connecting directly would require exposing database root credentials in client-side code, which is an extreme security flaw. Instead, the 3-tier model routes requests through an application tier (Python WSGI) using authenticated REST APIs.

Q2: How does your application protect against SQL Injection?

Answer: We use **Parameterized Queries** via `mysql.connector`. All user inputs are passed as separate parameter tuples into `cursor.execute(sql, (param1, param2))` with `%s` placeholders. The MySQL driver escapes and treats these values strictly as data literals, preventing malicious SQL code injection.

Q3: How are passwords stored and verified securely?

Answer: Passwords are never stored in plain text. We use **PBKDF2 with HMAC-SHA256** with 310,000 iterations and a 16-byte cryptographically secure random salt generated via Python's `secrets` module. When a user logs in, the entered password is salted and hashed with the same parameters and compared using constant-time comparison (`hmac.compare_digest`) to prevent timing attacks.

Q4: What is the role of Gunicorn and WSGI in your backend?

Answer: WSGI (Web Server Gateway Interface, PEP 3333) is the standard interface between Python web code and HTTP servers. Gunicorn is a production-grade WSGI HTTP server that implements a pre-fork worker model. It starts a master process that manages multiple worker processes (3 workers in our setup), enabling our application to handle concurrent HTTP requests without locking.

Q5: How does your hosting setup ensure that data is not lost when containers restart?

Answer: We use a **Persistent Volume** in Railway. While application containers are ephemeral and stateless, the MySQL service card has a cloud block volume mounted to `/var/lib/mysql`. All database tables, indexes, and transactions are physically written to this persistent volume, ensuring zero data loss during restarts or redeployments.

Q6: How do you enforce jurisdictional rules for agricultural officers?

Answer: Through database relational constraints and server-side authorization checks. Each Agricultural Officer belongs to an `Agriculture_Office` (Krishi Bhavan). Each Farmer belongs to a `Panchayath` linked to an office. When reviewing subsidies, the query checks `panchayaths.office_id = officers.office_id`. If an officer tries to review a farmer outside their jurisdiction, the backend rejects it with `HTTP 403 Forbidden`.

Q7: Why did you choose Railway and Docker for deployment?

Answer: Docker provides reproducibility across operating systems (running the exact same Linux environment in production as in development). Railway provides seamless micro-service orchestration, automated SSL certificate provisioning via Let's Encrypt, and private internal networking between the web tier and database tier.